# ATP Research

Automated Theorem Proving Platform

Technical Documentation

Version 2.0.0 — February 2026

Python (FastAPI) + C++ + Prover9 + AI Translation

https://atp-research.com/

# Table of Contents


1 Overview

2 Features

3 Architecture & Tech Stack

4 Installation & Setup

5 Project Structure

6 API Reference

7 Database Schema

8 How the Provers Work

9 Prover9 Syntax Reference

10 Pages & UI Guide

# 1. Overview

ATP Research is an interactive web application for **Automated Theorem Proving** (ATP). It provides three prover engines — propositional logic (C++), first-order logic (C++), and the real **Prover9** binary — along with AI-powered proof translation, a blog system, user authentication, and a mathematical textbook.

The application follows a **server/web** architecture: all computation (theorem proving, authentication, database operations) runs on the Python server, while the frontend is a pure static HTML/CSS/JS client.

### Key Highlights

- **Server-side proving** — All prover computation runs on the backend via C++ binaries and Prover9
- **AI Translation** — Prover9 proofs are automatically translated to human-readable Japanese by Claude AI
- **Three prover modes** — Propositional logic, first-order logic, and Prover9
- **Educational content** — 5-chapter mathematical textbook covering logic foundations

# 2. Features

### Three Prover Engines

Propositional (C++ resolution), FOL (C++ Herbrand/unification), and Prover9 (William McCune). All server-side.

### AI Proof Translation

Claude API translates Prover9 output into human-readable Japanese explanations, displayed alongside the raw proof.

### Chat Interface

Interactive chat-style prover on the Member page. Send formulas and receive step-by-step proofs.

### Blog System

Markdown-based blog with CRUD, tag categorization, and homepage preview. Members-only editing.

### Mathematical Textbook

5 chapters: Logic Symbols, Set Construction, Pairs & Functions, Algebraic Basics, Equivalence & Rationals.

### Auth & Theme

bcrypt password hashing, signed cookie sessions, role-based access. Dark/Light/System theme toggle.

# 3. Architecture & Tech Stack

## Architecture

```
Browser (web/) — HTML/CSS/JS
↕ fetch() API calls
FastAPI Server (server/) — Python
↕ subprocess / sqlite3
C++ Binaries | Prover9 | SQLite DB | Claude API
```

## Tech Stack

| Layer | Technology |
|---|---|
| Backend | Python 3.10+ / FastAPI / uvicorn |
| Provers | C++17 (clang++) — resolution.cpp, fol_prover.cpp |
| External Prover | Prover9 2009-11A (William McCune) |
| AI Translation | Anthropic Claude API (claude-haiku-4-5) |
| Database | SQLite (stdlib sqlite3, WAL mode) |
| Auth | bcrypt + itsdangerous (signed cookies) |
| Markdown | python-markdown |
| Frontend | Vanilla HTML / CSS / JavaScript |
| Fonts | Inter, JetBrains Mono, Noto Sans JP |

# 4. Installation & Setup

## Prerequisites

- **Python 3.10+**
- **C++ compiler** — clang++ or g++ with C++17 support
- **Prover9** — installed and available in `$PATH`
- (Optional) **Anthropic API key** — for AI proof translation

## Installing Prover9 (macOS)

```
brew install prover9
prover9 --version
```

## Setup

```
# Install Python dependencies
pip install -r server/requirements.txt

# Compile C++ prover binaries
make -C server/prover
```

## Running

```
# Basic
uvicorn main:app --host 0.0.0.0 --port 3333 --reload --app-dir server

# With AI translation
ANTHROPIC_API_KEY=sk-ant-... uvicorn main:app --host 0.0.0.0 --port 3333 --reload --app-dir
server
```

The server starts at **http://localhost:3333**.

## Default Credentials

**Username:** `admin` **Password:** `admin123`

The admin account and 5 sample blog posts are seeded on first run.

# 5. Project Structure

```
atp-research/
├── server/                    # Python (FastAPI) backend
│   ├── main.py                # App entry, routers, static serving
│   ├── auth.py                # Auth endpoints + signed cookies
│   ├── blog.py                # Blog CRUD endpoints
│   ├── prover_api.py          # Prover endpoints + AI translation
│   ├── database.py            # SQLite setup, schema, seeds
│   ├── requirements.txt       # Python dependencies
│   ├── atp.db                 # SQLite database (auto-created)
│   └── prover/                # C++ prover engine
│       ├── resolution.cpp     # Propositional: CNF → resolution
│       ├── fol_prover.cpp     # FOL: Skolem → Herbrand → unify
│       ├── Makefile           # Compile both binaries
│       ├── resolution         # Compiled binary
│       └── fol_prover         # Compiled binary
│
├── web/                       # Static frontend (pure client)
│   ├── index.html             # Landing page + prover demo
│   ├── member.html            # Chat-style prover (3 modes)
│   ├── textbook.html          # Mathematical textbook
│   ├── blog.html              # Blog listing
│   ├── blog-editor.html       # Blog editor (auth required)
│   ├── login.html             # Login / registration
│   ├── css/
│   │   ├── style.css          # Global + theme variables
│   │   ├── member.css         # Chat + AI explanation styles
│   │   ├── textbook.css       # Textbook + TOC
│   │   └── blog.css           # Blog pages
│   ├── js/
│   │   ├── api.js             # Centralized API client
│   │   ├── chat.js            # Chat UI → server APIs
│   │   ├── app.js             # Homepage UI → server API
│   │   └── theme.js           # Dark/Light/System toggle
│   └── images/
│
├── README.md
└── docs-print.html            # This documentation (PDF source)
```

# 6. API Reference

## Prover Endpoints

| Method | Endpoint | Body | Description |
|---|---|---|---|
| `POST` | `/api/prove/propositional` | `{ formula }` | Propositional proof (C++) |
| `POST` | `/api/prove/fol` | `{ formula }` | First-order logic proof (C++) |
| `POST` | `/api/prove/prover9` | `{ goals, formulas?, timeout? }` | Prover9 + AI translation |

### Prover9 Request

```
{
  "goals": "x = x.",
  "formulas": "(optional assumptions)",
  "timeout": 10
}
```

### Prover9 Response

```
{
  "success": true,
  "proved": true,
  "proof": "% Proof 1 at 0.00 seconds...",
  "stats": "User_CPU=0.00",
  "output": "(full Prover9 output)",
  "input": "(generated input file)",
  "aiExplanation": "(AI translation in Japanese, or null)"
}
```

## Authentication Endpoints

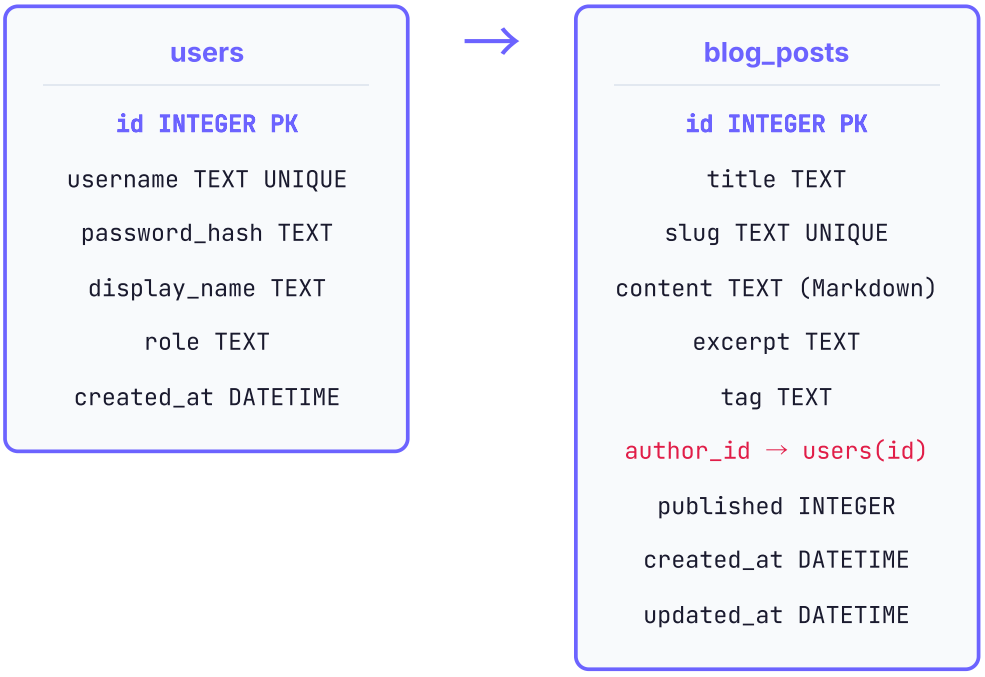
| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/api/auth/me` | Get current logged-in user |
| `POST` | `/api/auth/login` | Login with `{ username, password }` |
| `POST` | `/api/auth/register` | Register with `{ username, password, display_name }` |
| `POST` | `/api/auth/logout` | Logout (destroys session) |

## Blog Endpoints

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| `GET` | `/api/blog?limit=4` | No | List published posts |
| `GET` | `/api/blog/{slug}` | No | Get post by slug |
| `POST` | `/api/blog` | Yes | Create new post |
| `PUT` | `/api/blog/{id}` | Yes | Update post |
| `DELETE` | `/api/blog/{id}` | Yes | Delete post |

# 7. Database Schema

## ER Diagram




## SQL Definitions

```
CREATE TABLE users (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  username      TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  display_name  TEXT NOT NULL,
  role          TEXT DEFAULT 'member',
  created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE blog_posts (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT NOT NULL,
  slug        TEXT UNIQUE NOT NULL,
  content     TEXT NOT NULL,
  excerpt     TEXT,
  tag         TEXT DEFAULT 'General',
  author_id   INTEGER REFERENCES users(id),
  published   INTEGER DEFAULT 1,
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

# 8. How the Provers Work

## Propositional Logic (C++ — resolution.cpp)

1. **Tokenize** — Lex the input into tokens (operators, variables, parens)
2. **Parse** — Recursive-descent parser builds an AST with correct precedence
3. **Negate** — Assume the negation of the formula (refutation method)
4. **CNF** — Convert to Conjunctive Normal Form:
   - Eliminate ↔ and →
   - Push negation inward (De Morgan's laws)
   - Distribute ∨ over ∧
5. **Resolve** — Apply the resolution rule on complementary literals (max 5000 iterations)
6. **Result** — If the empty clause □ is derived → formula is a tautology

## First-Order Logic (C++ — fol_prover.cpp)

1. **Parse** — FOL formula with quantifiers ( `all x` , `exists x` ) and predicates
2. **NNF** — Negation Normal Form conversion
3. **Skolemize** — Replace ∃ with Skolem functions/constants
4. **CNF** — Convert to clause form
5. **Herbrand instantiation** — Ground the formula with terms from the Herbrand universe
6. **Unification** — Robinson's unification algorithm with occurs check
7. **Resolve** — Apply resolution on unifiable complementary literals

## Prover9 (System Binary + AI Translation)

1. User input is wrapped in `formulas(goals). ... end_of_list.`
2. Written to a temporary file
3. The `prover9` binary is executed with configurable timeout (max 30s)
4. Output is parsed for `THEOREM PROVED` / `SEARCH FAILED`
5. Proof steps and statistics are extracted
6. If `ANTHROPIC_API_KEY` is set, Claude AI translates the proof into Japanese

# 9. Prover9 Syntax Reference

| Operator | Syntax | Example |
|---|---|---|
| Universal quantification | `all x` | `all x P(x)` |
| Existential quantification | `exists x` | `exists x P(x)` |
| Negation | `-` | `-P(x)` |
| Conjunction (AND) | `&` | `P(x) & Q(x)` |
| Disjunction (OR) | `\|` | `P(x) \| Q(x)` |
| Implication | `→` | `P(x) → Q(x)` |
| Biconditional | `⟷` | `P(x) ⟷ Q(x)` |
| Equality | `=` | `x = y` |

## Example Formulas

```
# Identity
x = x.

# Modus Ponens
(all x (P(x) → Q(x))) & P(a) → Q(a).

# Symmetry of equality
x = y → y = x.

# Group theory: left identity + left inverse → right identity
(all x (e * x = x)) & (all x (i(x) * x = e)) → (all x (x * e = x)).
```

**Note:** Every Prover9 formula must end with a period ( `.` ).

# 10. Pages & UI Guide

| Page | URL | Description |
|---|---|---|
| Homepage | `/` | Landing page with hero, propositional prover demo, theorem cards, blog preview |
| Member | `/member` | Chat-style prover (Prover9 / Propositional / FOL modes), example buttons |
| Textbook | `/textbook` | 5-chapter mathematical textbook with sidebar TOC |
| Blog | `/blog` | All published blog posts with tag filtering |
| Blog Editor | `/blog-editor` | Create/edit/delete posts (requires login) |
| Login | `/login` | Login and registration forms |

## Theme System

Every page includes a theme toggle button (top-right) with three modes: **Dark**, **Light**, and **System**. The preference is persisted in `localStorage` and applied instantly without a flash of unstyled content.

## Navigation

The header contains links to all pages. On mobile, a hamburger menu replaces the horizontal navigation. Auth-dependent links (Member, Blog Editor) are shown/hidden based on session state.

ATP Research — Technical Documentation v2.0.0
Generated February 2026
Python (FastAPI) + C++17 + Prover9 + Claude AI